

FIAP ON

09

INTELIGÊNCIA NA BORDA: FOG E EDGE COMPUTING TRANSFORMANDO A IOT

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Representação das camadas de processamento em uma arquitetura IoT com Cloud, Fog e Edge Computing 8
Figura 2 - Arquitetura conceitual da computação em névoa (fog computing) aplicada à IoT 12
Figura 3 - Arquitetura da experiência de Fog Computing.....13
Figura 4 - Ligação entre ESP32 DevKit V4 e sensor MPU6050 via I²C 15
Figura 5 - Fluxo de processamento local no Node-RED 18
Figura 6 - Configuração do nó MQTT de entrada no Node-RED 19
Figura 7 - Indicador visual de movimento no dashboard do Node-RED.....20
Figura 8 - Fluxo no Node-RED para visualização mínima dos eventos de movimento 28

FIAP

LISTA DE QUADROS

Quadro 1 - Conexões entre ESP32 e MPU6050 com uso do pino de interrupção 24
Quadro 2 - Comparação objetiva entre Cloud, Fog e Edge Computing 33
Quadro 3 - Comparativo técnico entre Cloud, Fog e Edge Computing nos aspectos de consumo de energia, latência e robustez 35
Quadro 4 - Critérios de seleção e recomendação de abordagem predominante 37

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Verificação do serviço Mosquitto no Windows.....16
Código-fonte 2 - Instalação do Mosquitto e clientes no Linux (Ubuntu/Debian) 16
Código-fonte 3 - Execução do Mosquitto via Docker.....16
Código-fonte 4 - Configuração da conexão MQTT com o broker Mosquitto local 17
Código-fonte 5 - Inicialização do sensor MPU6050 com interrupção configurada 25
Código-fonte 6 - Função de interrupção para tratamento do evento 26
Código-fonte 7 - Loop principal com lógica de decisão 26
Código-fonte 8 - Publicação MQTT somente após evento de aceleração 27

FIAP

SUMÁRIO

1 INTELIGÊNCIA NA BORDA: FOG E EDGE COMPUTING TRANSFORMANDO A IOT 6
1.1 Computação distribuída em IoT: Conceito, papel e características 6
1.2 Visão geral das arquiteturas descentralizadas na IoT 7
1.2.1 Centralização vs. processamento local: diferenças práticas 9
1.2.2 Vantagens e limitações das abordagens distribuídas 10

2 FOG COMPUTING 11
2.1 Fundamentos da computação em névoa para sistemas IoT 11
2.2 Experiência prática 13
2.2.1 Hardware – ESP32 e MPU6050 14
2.2.2 Transmissão MQTT – Mosquitto Broker Local 16
2.2.3 Processamento e lógica local – Node-RED 17
2.2.4 Visualização – Dashboard interativo no Node-RED 19

3 EDGE COMPUTING 21
3.1 Fundamentos da inteligência na borda e microcontroladores inteligentes 21
3.2 Experiência prática 22
3.2.1 Hardware – ESP32 e MPU6050 com interrupção no pino INT 23
3.2.2 Processamento embarcado – Filtragem e tomada de decisão no firmware 24
3.2.3 Comunicação eficiente – Envio apenas de eventos relevantes via MQTT 27
3.2.4 Visualização – Interface mínima com Node-RED 28

4 COMPARAÇÃO ENTRE NUVEM, NÉVOA E BORDA 30
4.1 Matriz comparativa detalhada 31
4.2 Análise de consumo de energia, latência e robustez 33
4.3 Critérios para escolha da abordagem ideal 35

REFERÊNCIA 38

FIAP

1 INTELIGÊNCIA NA BORDA: FOG E EDGE COMPUTING TRANSFORMANDO A IOT

1.1 Computação distribuída em IoT: Conceito, papel e características

A computação distribuída em Internet das Coisas (IoT) é um paradigma no qual o processamento, o armazenamento e a análise de dados são realizados de forma descentralizada, aproveitando recursos distribuídos entre dispositivos de borda, nós intermediários e a nuvem. Diferente de um modelo estritamente centralizado, no qual todos os dados precisam ser enviados a um data center remoto para processamento, a computação distribuída permite que parte significativa desse trabalho ocorra mais próximo da fonte de geração de dados.

Em um ambiente IoT, sensores, atuadores e controladores estão frequentemente espalhados por grandes áreas geográficas, coletando dados em tempo real sobre variáveis físicas, químicas ou ambientais. A transmissão de todos esses dados brutos para a nuvem pode acarretar problemas de latência, consumo de largura de banda e até riscos de indisponibilidade em situações críticas. A computação distribuída surge como resposta a esses desafios, permitindo um balanceamento inteligente entre o processamento local e remoto.

Além de reduzir a sobrecarga na infraestrutura de rede, esse modelo possibilita respostas mais rápidas, uma vez que as decisões críticas podem ser tomadas diretamente na borda ou em níveis intermediários, sem depender exclusivamente da conectividade com a nuvem. Em aplicações como veículos autônomos, sistemas industriais e monitoramento de saúde, essa característica é fundamental para garantir segurança, confiabilidade e continuidade operacional.

Do ponto de vista arquitetural, a computação distribuída na IoT se apoia em três camadas principais:

- **Borda (Edge Computing):** nível mais próximo do dispositivo ou sensor, responsável por processar dados localmente e enviar apenas informações relevantes ou eventos detectados.

- **Névoa (Fog Computing):** camada intermediária que conecta a borda à nuvem, atuando como ponto de processamento regional, filtragem de dados e integração com sistemas locais.
- **Nuvem (Cloud Computing):** camada centralizada que oferece grande capacidade de armazenamento e processamento para análises avançadas e integração com serviços de alto nível.

Essa divisão hierárquica não é rígida, e a alocação das funções de cada camada depende das características do sistema, dos requisitos de desempenho e da disponibilidade de recursos. Em projetos reais, a definição da estratégia de computação distribuída é determinante para atingir equilíbrio entre desempenho, custo e robustez, tornando-se uma etapa essencial no planejamento de soluções IoT.

Compreender o papel dessas arquiteturas descentralizadas e como elas se complementam é o próximo passo para projetar sistemas de IoT eficientes. Na seção a seguir, veremos uma visão geral das arquiteturas descentralizadas, explorando seus elementos, interações e relevância para diferentes cenários de aplicação.

1.2 Visão geral das arquiteturas descentralizadas na IoT

A computação distribuída em IoT refere-se à organização de sistemas nos quais o processamento, o armazenamento e a tomada de decisão não estão concentrados em um único ponto, mas distribuídos entre múltiplos dispositivos e camadas da arquitetura. Essa abordagem surgiu como resposta à crescente demanda por aplicações mais rápidas, escaláveis e resilientes, capazes de lidar com o grande volume de dados gerado por sensores, atuadores e outros dispositivos conectados.

Em vez de transmitir todas as informações brutas diretamente para um servidor central, parte do processamento é executada mais próxima da origem dos dados, o que reduz a latência, diminui o consumo de banda e aumenta a autonomia do sistema. O papel da computação distribuída é fundamental para viabilizar aplicações críticas, como sistemas de monitoramento industrial, veículos autônomos e redes inteligentes de energia, onde a velocidade de resposta e a confiabilidade são determinantes.

A figura a seguir ilustra a relação entre as três camadas mais comuns nesse contexto: a **borda** (edge), a **névoa** (fog) e a **nuvem** (cloud). Na camada inferior, encontram-se os dispositivos de borda e seus microcontroladores, responsáveis pelo

processamento local imediato. No nível intermediário, a camada de névoa agrega, filtra e pré-processa os dados recebidos da borda. Na camada superior, a nuvem realiza análises históricas, armazena grandes volumes de dados e integra informações com serviços e plataformas externas.

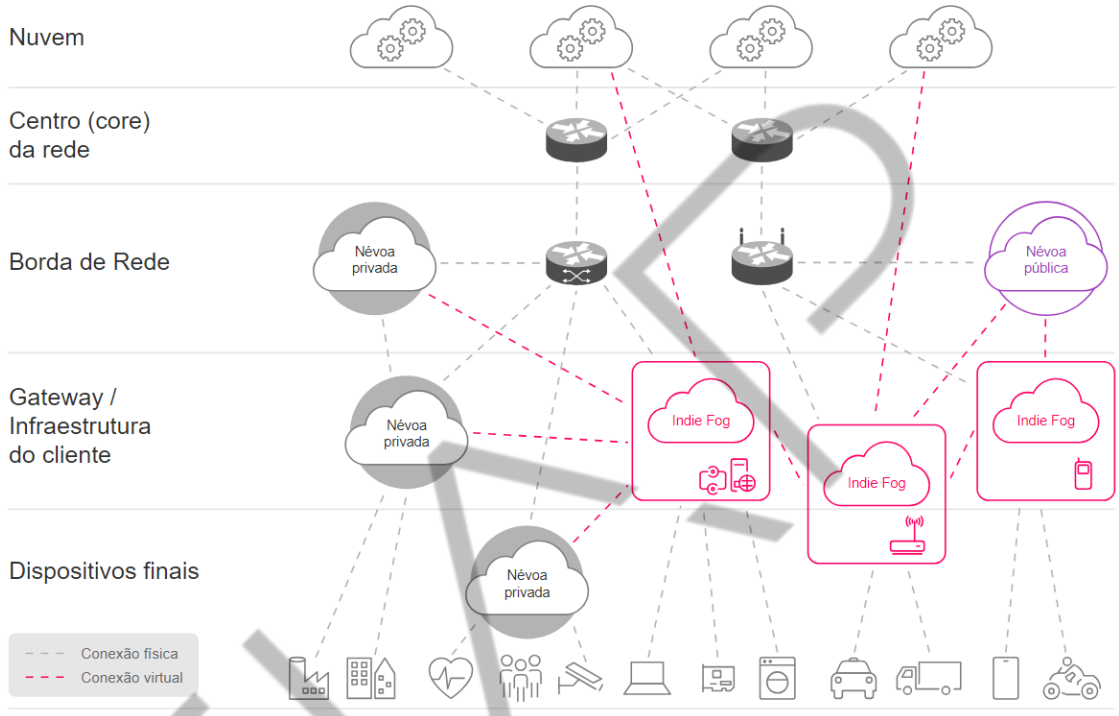


Figura 1 - Representação das camadas de processamento em uma arquitetura IoT com Cloud, Fog e Edge Computing
Fonte: Oliveira (2019), adaptado por FIAP (2025)

As principais características dessa abordagem incluem a escalabilidade horizontal, que permite a adição de novos nós conforme a demanda aumenta; a resiliência a falhas, garantindo continuidade mesmo em condições adversas; e a flexibilidade na integração de diferentes tecnologias e protocolos. No entanto, também há desafios a serem superados, como a complexidade na coordenação entre múltiplos nós, a necessidade de mecanismos de segurança distribuídos e a padronização das interfaces de comunicação.

Essa base conceitual serve como ponto de partida para compreender como diferentes paradigmas de processamento descentralizado são aplicados na prática. Nas próximas subseções, será explorado com mais detalhes o papel específico dessas arquiteturas e como elas influenciam o desempenho, a segurança e a eficiência dos sistemas conectados.

1.2.1 Centralização vs. processamento local: diferenças práticas

A partir da representação da figura “Representação das camadas de processamento em uma arquitetura IoT com Cloud, Fog e Edge Computing”, é possível compreender que a principal distinção entre uma arquitetura centralizada e uma distribuída está no local onde ocorre o processamento e a tomada de decisão. Na abordagem centralizada, todos os dados coletados pelos dispositivos são enviados integralmente para um servidor remoto, normalmente na nuvem, onde são processados e analisados. Essa estratégia simplifica a gestão e o controle, pois as regras de negócio, a lógica de análise e o armazenamento estão concentrados em um único ponto.

Entretanto, essa centralização gera impactos diretos na performance e na resiliência do sistema. Como todo o tráfego precisa percorrer o caminho até a nuvem e retornar, há maior latência na resposta e maior dependência de conectividade estável. Em cenários críticos — como monitoramento de equipamentos industriais ou detecção de incidentes de segurança —, esses milissegundos adicionais podem representar risco à operação.

No modelo com processamento local, característico de arquiteturas de borda e névoa, parte da análise é realizada próxima à origem dos dados. Isso permite que decisões imediatas sejam tomadas sem a necessidade de aguardar a resposta de um servidor distante. Por exemplo, um sensor de vibração em uma máquina industrial pode acionar um alarme local ao detectar padrões que indiquem falha iminente, enquanto envia dados agregados para análise histórica na nuvem.

A escolha entre centralização e processamento local não é obrigatoriamente exclusiva. Muitos sistemas adotam modelos híbridos, nos quais tarefas críticas são tratadas localmente e funções de planejamento e análise de longo prazo são executadas na nuvem. Essa divisão de responsabilidades equilibra a necessidade de velocidade com o aproveitamento de recursos computacionais de alto desempenho.

Compreender essa diferença é fundamental para projetar arquiteturas IoT eficientes, pois influencia diretamente fatores como consumo de energia, custo de transmissão, segurança e escalabilidade. A próxima subseção apresentará com mais detalhes as vantagens e limitações das abordagens distribuídas no contexto IoT.

1.2.2 Vantagens e limitações das abordagens distribuídas

A adoção de arquiteturas distribuídas em IoT traz benefícios claros em termos de desempenho, escalabilidade e resiliência. Um dos principais pontos positivos é a redução da latência, já que o processamento é realizado próximo à origem dos dados, permitindo respostas quase imediatas. Esse fator é especialmente importante em aplicações críticas, como sistemas de saúde, controle de tráfego e automação industrial, onde atrasos na tomada de decisão podem gerar consequências graves.

Outro benefício é a diminuição da carga de transmissão de dados para a nuvem. Ao processar e filtrar informações localmente, apenas os dados relevantes ou agregados são enviados, reduzindo custos com banda e armazenamento e contribuindo para maior eficiência energética. Além disso, a descentralização aumenta a resiliência do sistema, pois eventuais falhas em um servidor central não interrompem totalmente a operação.

A escalabilidade também se torna mais simples, visto que novos dispositivos ou nós de processamento podem ser adicionados de forma modular para atender ao crescimento da demanda. Essa característica permite expandir a infraestrutura gradualmente, sem a necessidade de substituições completas ou reestruturações profundas.

Por outro lado, há desafios que precisam ser considerados. A gestão e a coordenação entre múltiplos nós tornam-se mais complexas, exigindo protocolos eficientes para sincronização e consistência dos dados. A segurança também requer atenção especial, pois a superfície de ataque aumenta à medida que mais pontos de processamento e comunicação são incorporados à rede. Além disso, a padronização de interfaces e protocolos é essencial para garantir interoperabilidade entre diferentes fabricantes e plataformas, o que nem sempre é simples de implementar.

Portanto, a escolha por uma arquitetura distribuída deve considerar não apenas os ganhos de desempenho e autonomia, mas também o impacto na complexidade do gerenciamento e nos requisitos de segurança. O equilíbrio entre esses fatores será determinante para o sucesso do projeto, especialmente em ambientes críticos e de larga escala.

2 FOG COMPUTING

A computação em névoa (Fog Computing), representa uma extensão da computação em nuvem, trazendo parte do processamento para mais próximo da borda da rede. Ao adotar esse modelo, é possível reduzir a latência, diminuir a quantidade de dados trafegados para a nuvem e permitir respostas mais rápidas e localizadas. Essa abordagem é particularmente relevante para aplicações em tempo real e em ambientes com conectividade limitada ou intermitente.

Diferente da Cloud Computing, em que todo o processamento ocorre em servidores remotos, na Fog Computing há um compartilhamento entre dispositivos intermediários, como gateways, roteadores, computadores locais ou servidores regionais. Esses dispositivos processam os dados de forma parcial ou total, antes de decidir se devem ou não ser encaminhados para a nuvem.

Na área de IoT, essa descentralização é especialmente importante em sistemas críticos, como monitoramento industrial, veículos autônomos, cidades inteligentes e aplicações sensíveis ao tempo de resposta. Por exemplo, em um sistema de segurança predial, a identificação de uma anomalia deve ocorrer localmente, sem depender do tempo de envio e retorno de dados de um servidor em nuvem.

A próxima subseção apresentará uma experiência prática que exemplifica a utilização da arquitetura fog para processar dados de sensores em tempo real com o ESP32 e o acelerômetro MPU6050. O objetivo é demonstrar como é possível realizar a detecção local de movimentos diretamente no Node-RED, utilizando lógica programável e dashboards interativos.

2.1 Fundamentos da computação em névoa para sistemas IoT

A computação em névoa é uma extensão da computação em nuvem que leva parte do processamento e armazenamento para o mais próximo da borda da rede, entre os dispositivos finais e a nuvem. O principal objetivo da computação em névoa é reduzir a latência, economizar largura de banda e aumentar a resiliência dos sistemas IoT, mantendo parte da inteligência em um nível intermediário entre a borda e a nuvem.

Em vez de enviar todos os dados brutos dos sensores diretamente para a nuvem, a arquitetura de Fog Computing permite que esses dados sejam parcialmente processados localmente em servidores intermediários, gateways ou computadores próximos ao ambiente monitorado. Isso reduz o tempo de resposta e evita sobrecarga na infraestrutura de nuvem, sendo especialmente útil em aplicações que exigem decisões em tempo real, como automação industrial, veículos autônomos, cidades inteligentes e monitoramento de desastres naturais.

A figura “Arquitetura conceitual da computação em névoa (fog computing) aplicada à IoT” ilustra esse conceito, evidenciando a posição estratégica da camada de névoa entre os sensores e a nuvem. Nessa arquitetura, os dados coletados pelos sensores são processados em nível intermediário antes de serem enviados à nuvem para armazenamento ou análise mais aprofundada.

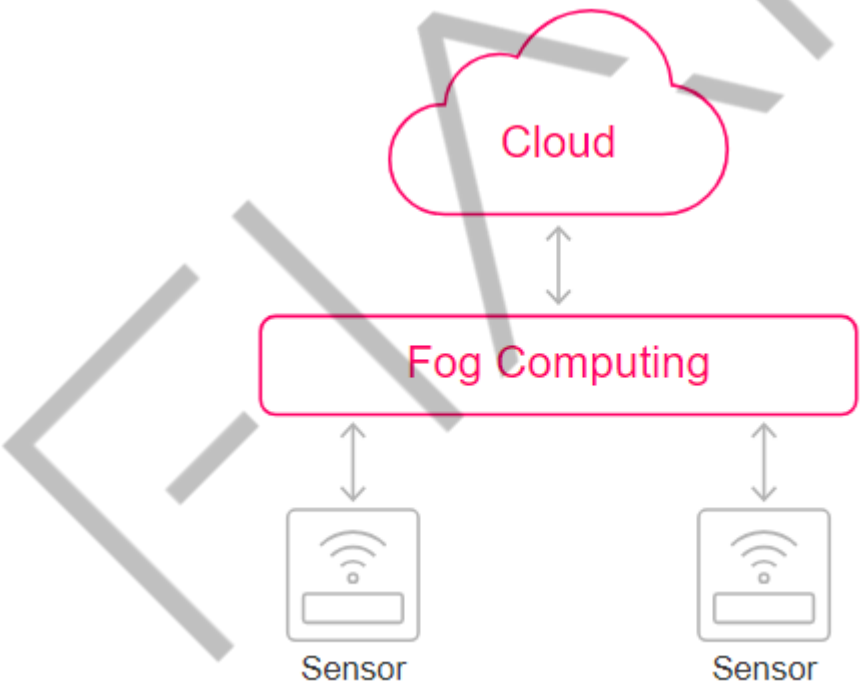


Figura 2 - Arquitetura conceitual da computação em névoa (fog computing) aplicada à IoT
Fonte: Elaborado pelo autor (2025)

Com esses fundamentos teóricos em mente, a próxima subseção apresenta uma experiência prática de computação em névoa utilizando ESP32, o sensor MPU6050 e o ambiente de fluxo Node-RED para tomada de decisão local em tempo real.

2.2 Experiência prática

Nesta experiência, será demonstrado como aplicar os conceitos de Fog Computing utilizando um microcontrolador ESP32 com o sensor MPU6050, o broker MQTT Mosquitto instalado localmente e o ambiente de desenvolvimento Node-RED. O objetivo é processar os dados de movimento localmente, sem a dependência de serviços em nuvem, evidenciando as vantagens da computação em névoa, como baixa latência, maior controle sobre os dados e independência de conectividade externa.

A arquitetura da solução é apresentada na figura a seguir, que ilustra a sequência de etapas: a coleta dos dados brutos de aceleração pelo ESP32 conectado ao MPU6050, o envio das informações por meio do protocolo MQTT ao broker Mosquitto configurado na máquina local, o recebimento e o processamento desses dados pelo Node-RED e, por fim, a tomada de decisão, como o acionamento de um LED em resposta a padrões identificados.

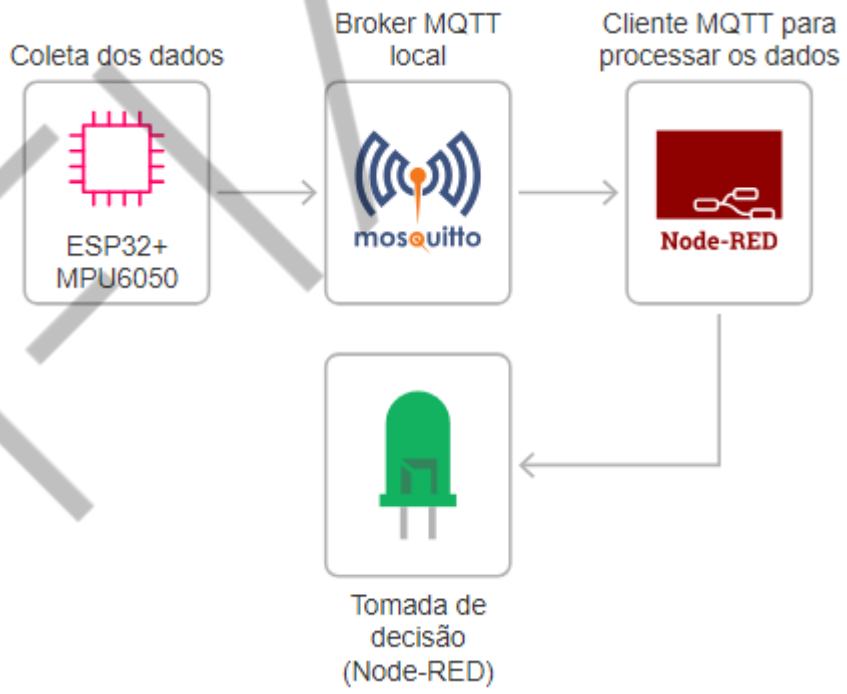


Figura 3 - Arquitetura da experiência de Fog Computing
Fonte: Elaborado pelo autor (2025)

Essa abordagem é particularmente vantajosa para aplicações que exigem resposta rápida e não podem depender da latência de redes externas. Além disso,

permite que o sistema continue funcionando mesmo em ambientes sem acesso à internet, reforçando a robustez da solução.

A implementação prática será detalhada nas próximas subseções, abordando desde o hardware necessário até a criação de dashboards no Node-RED com lógica de decisão embarcada.

2.2.1 Hardware – ESP32 e MPU6050

Nesta etapa prática de Fog Computing, o conjunto de hardware utilizado é composto pela placa ESP32 DevKit V4 e pelo sensor inercial MPU6050, interligados por meio do barramento I²C. Essa configuração é responsável por coletar dados de aceleração nos eixos X, Y e Z, essenciais para aplicações de detecção de movimento, análise de padrões e automação em sistemas embarcados.

A comunicação I²C garante simplicidade na conexão e eficiência na troca de dados, utilizando apenas dois fios para transmissão (SDA e SCL) além das linhas de alimentação (VCC e GND). No caso do ESP32, a definição dos pinos de SDA e SCL é flexível, permitindo adequações conforme o projeto.

A próxima figura apresenta o diagrama de ligação entre o ESP32 e o MPU6050, destacando as conexões elétricas necessárias para a operação correta do conjunto.

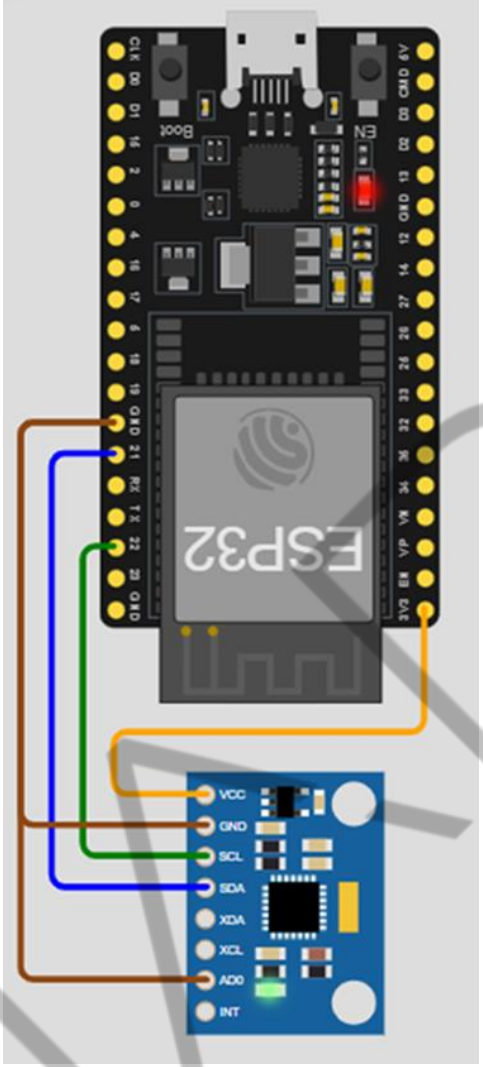


Figura 4 - Ligação entre ESP32 DevKit V4 e sensor MPU6050 via I²C
Fonte: Elaborado pelo autor (2025)

- VCC do MPU6050 é conectado ao pino 3V3 do ESP32, garantindo alimentação estável;
- GND do MPU6050 é conectado ao GND do ESP32;
- SDA (linha de dados) é conectado ao GPIO 21;
- SCL (linha de clock) é conectado ao GPIO 22;
- O pino INT do MPU6050 pode ser utilizado para gerar interrupções, otimizando o processamento.

Essa base de hardware é fundamental para implementar testes de computação em névoa, permitindo que a coleta e o pré-processamento dos dados ocorram localmente antes do envio para outros sistemas ou para a nuvem.

2.2.2 Transmissão MQTT – Mosquitto Broker Local

Nesta experiência, os dados do sensor MPU6050 são transmitidos do ESP32 para o ambiente de processamento utilizando o protocolo MQTT, porém com um broker local em vez de uma solução em nuvem. Essa estratégia se alinha ao conceito de Fog Computing, ao aproximar o processamento dos dados do local de coleta, reduzindo a latência e aumentando a autonomia da solução.

Foi utilizado o Mosquitto, uma implementação leve e gratuita do protocolo MQTT, amplamente adotada em aplicações de Internet das Coisas. A execução do broker pode ser realizada diretamente em uma máquina local com Windows, Linux ou via Docker.

Para verificar se o Mosquitto está instalado corretamente no Windows e identificar se o serviço foi inicializado com sucesso, pode-se utilizar o seguinte comando:

```
mosquitto -v
```

Código-fonte 1 - Verificação do serviço Mosquitto no Windows
Fonte: Elaborado pelo autor (2025)

Em sistemas baseados em Linux, como o Ubuntu, a instalação do Mosquitto e de seus clientes pode ser realizada via terminal com os comandos a seguir:

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Código-fonte 2 - Instalação do Mosquitto e clientes no Linux (Ubuntu/Debian)
Fonte: Elaborado pelo autor (2025)

Outra alternativa prática e multiplataforma é executar o Mosquitto por meio de um contêiner Docker, ideal para quem já possui o Docker instalado:

```
docker run -it -p 1883:1883 eclipse-mosquitto
```

Código-fonte 3 - Execução do Mosquitto via Docker
Fonte: Elaborado pelo autor (2025)

Com o broker iniciado, é necessário alterar o código do ESP32 para se conectar ao IP da máquina local onde o Mosquitto está rodando. Essa modificação permite que o microcontrolador publique os dados diretamente na rede local.

```
const char* mqtt_server = "192.168.0.10"; // IP da máquina
com Mosquitto local


WiFiClient espClient;
PubSubClient client(espClient);


void reconnect() {
  while (!client.connected()) {
    if (client.connect("ESP32Client")) {
      Serial.println("Conectado ao Mosquitto local!");
    } else {
      delay(5000);
    }
  }
}
```

Código-fonte 4 - Configuração da conexão MQTT com o broker Mosquitto local
Fonte: Elaborado pelo autor (2025)

O código-fonte anterior utiliza a biblioteca PubSubClient para gerenciar a comunicação MQTT. A função reconnect() garante que a conexão seja restabelecida automaticamente caso haja queda temporária do broker ou falha de rede.

Ao configurar o ESP32 com um broker local, o sistema torna-se mais resiliente a falhas de internet, o que é fundamental em ambientes industriais, agrícolas ou locais remotos. A próxima subseção mostrará como utilizar o Node-RED localmente para receber e processar esses dados, completando o fluxo de uma arquitetura fog.

2.2.3 Processamento e lógica local – Node-RED

Uma das vantagens da computação em névoa é permitir que a lógica de tomada de decisão ocorra em um nível intermediário entre os dispositivos e a nuvem. Neste experimento, essa lógica é implementada no Node-RED, instalado localmente no computador que também hospeda o broker MQTT Mosquitto.

Na figura “Fluxo de processamento local no Node-RED”, observa-se o fluxo construído no Node-RED. O nó “Dados Crus (g)” realiza a subscrição no tópico sensor/movimento/fog, recebendo as leituras do acelerômetro MPU6050. Em seguida, os dados são encaminhados ao nó “Detecta Movimento (1.2g)”, responsável por processar o valor de aceleração e verificar se ultrapassa o limite estabelecido de 1.2 g, indicando uma possível movimentação brusca.

Caso a condição seja verdadeira, o fluxo ativa o nó “LED Movimento (g)”, que envia um comando MQTT para acionar o LED verde conectado ao ESP32, permitindo assim uma reação local e imediata ao evento detectado.



Figura 5 - Fluxo de processamento local no Node-RED
Fonte: NODE-RED (2025)

Para configurar o nó de entrada MQTT, basta arrastar o nó mqtt in até o fluxo e editar suas propriedades conforme apresentado na figura “Configuração do nó MQTT de entrada no Node-RED”. A configuração utiliza o broker local e assina o tópico sensor/movimento/fog, garantindo que os dados provenientes do ESP32 sejam corretamente recebidos e processados.

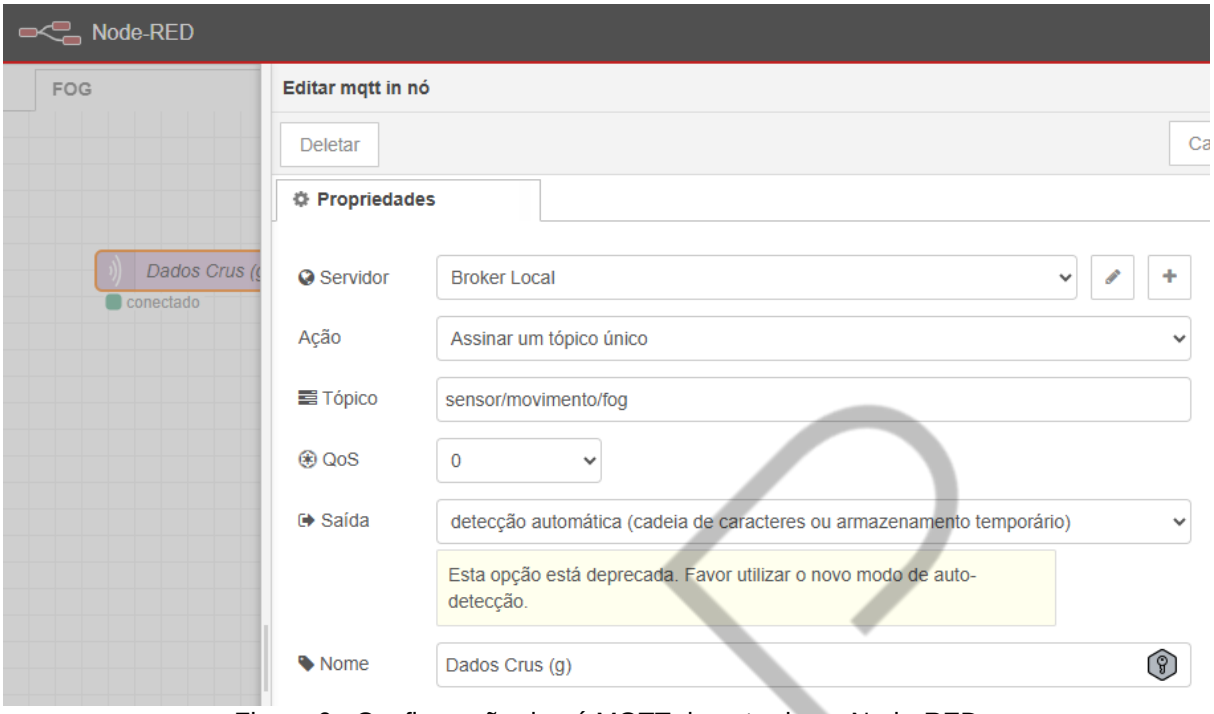


Figura 6 - Configuração do nó MQTT de entrada no Node-RED
Fonte: NODE-RED (2025)

Essa abordagem representa um modelo típico de fog computing, onde a lógica embarcada no fluxo do Node-RED permite decisões rápidas e descentralizadas, sem depender da conectividade com a nuvem. A próxima subseção abordará como visualizar os dados e as decisões tomadas por meio de um painel local no próprio Node-RED.

2.2.4 Visualização – Dashboard interativo no Node-RED

Após o processamento dos dados de aceleração bruta no fluxo do Node-RED, é essencial que o resultado seja apresentado de maneira clara para o usuário. A visualização local por meio de dashboards no próprio Node-RED permite acompanhar os eventos detectados em tempo real, oferecendo uma interface amigável para testes e validações.

No exemplo desta experiência, utilizou-se um indicador visual do tipo LED para representar a presença ou ausência de movimento, conforme ilustrado na figura “Indicador visual de movimento no dashboard do Node-RED”. Quando o valor de aceleração ultrapassa o limiar de 1.2g, o LED virtual acende na cor verde, indicando que um movimento significativo foi detectado. Caso contrário, o indicador permanece apagado (cinza).

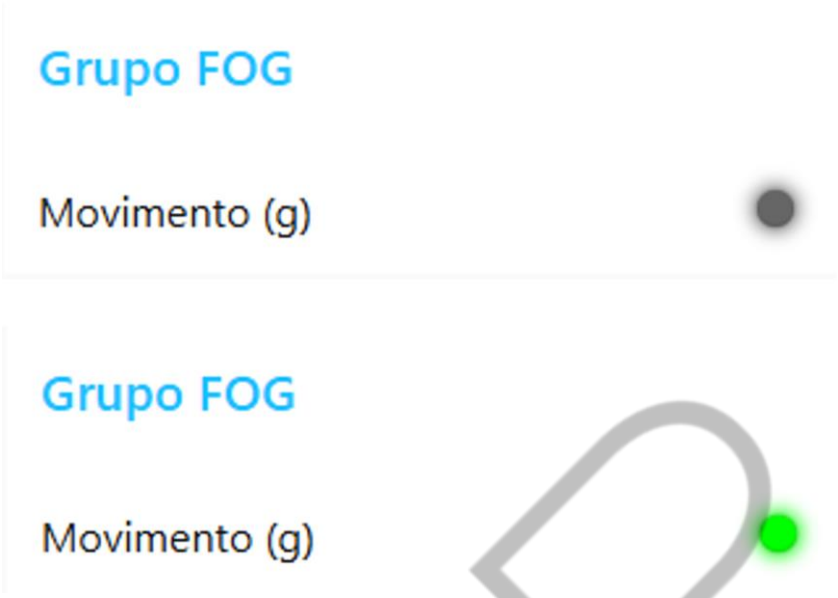


Figura 7 - Indicador visual de movimento no dashboard do Node-RED
Fonte: NODE-RED (2025)

Este painel está incluído no grupo chamado Grupo FOG, com o campo rotulado como Movimento (g). Para montar essa visualização, é necessário utilizar o nó ui_led do pacote node-red-contrib-ui-led. O valor lógico que aciona o LED é resultado do bloco function, que define a lógica de movimentação com base no valor absoluto da aceleração.

Essa abordagem demonstra como o Node-RED pode ser utilizado não apenas para processar dados localmente, mas também para apresentar resultados de maneira compreensível e responsiva, reforçando o papel da computação em névoa no contexto da Internet das Coisas.

Essa experiência demonstrou como a computação em névoa pode ser eficaz ao descentralizar o processamento de dados, oferecendo respostas rápidas por meio de lógica local no Node-RED. No entanto, em cenários ainda mais exigentes quanto à latência ou à autonomia de funcionamento, é possível levar esse processamento diretamente para o dispositivo embarcado, sem depender de softwares externos. A próxima seção explora justamente essa abordagem, por meio da computação na borda.

3 EDGE COMPUTING

A computação na borda, conhecida como Edge Computing, refere-se ao processamento de dados realizado diretamente no dispositivo onde os dados são gerados, sem depender de servidores locais ou em nuvem para a tomada de decisão. Em sistemas IoT, isso significa que sensores, microcontroladores e atuadores passam a desempenhar um papel mais ativo, executando algoritmos e lógica embarcada.

Essa abordagem visa reduzir a latência, aumentar a autonomia dos dispositivos e garantir maior robustez operacional, especialmente em ambientes com conectividade limitada ou em aplicações críticas, como sistemas de monitoramento industrial, dispositivos médicos portáteis ou veículos autônomos.

No contexto deste capítulo, a computação na borda será explorada utilizando o ESP32 com o sensor MPU6050, onde o próprio microcontrolador realizará a filtragem dos dados brutos de aceleração e identificará eventos de movimento. Apenas informações relevantes serão enviadas via MQTT, tornando o processo mais eficiente em termos de largura de banda e consumo energético.

A seguir, será apresentada uma experiência prática que ilustra essa arquitetura embarcada.

3.1 Fundamentos da inteligência na borda e microcontroladores inteligentes

A computação na borda representa uma evolução no paradigma dos sistemas distribuídos, ao permitir que os próprios dispositivos terminais realizem tarefas computacionais, muitas vezes reservadas a servidores centrais ou intermediários. Nessa abordagem, sensores e microcontroladores não apenas coletam e transmitem dados, mas também executam algoritmos para filtragem, classificação, decisão e atuação local.

Microcontroladores como o ESP32, Raspberry Pi Pico W ou Arduino Nano 33 BLE Sense têm se destacado por sua capacidade de executar modelos de aprendizado de máquina embarcados (TinyML), processar sinais em tempo real, realizar detecção de eventos e controlar dispositivos de forma autônoma.

Entre as principais vantagens da inteligência na borda, destacam-se:

- **Baixa latência:** decisões são tomadas localmente, sem a necessidade de envio dos dados para servidores ou nuvem;
- **Redução do tráfego de dados:** apenas eventos ou informações processadas são transmitidos;
- **Maior resiliência:** o sistema pode continuar operando mesmo com perda de conectividade;
- **Eficiência energética:** o envio de dados é otimizado, o que reduz o consumo de energia em sistemas alimentados por bateria.

Para que essa abordagem seja viável, é necessário utilizar dispositivos com maior poder computacional e memória, além de técnicas de otimização de código e modelos de IA compactos. Nos próximos tópicos, será detalhada uma experiência com o ESP32 e o sensor MPU6050, demonstrando como configurar o firmware para realizar a detecção de movimento localmente, utilizando o pino de interrupção para tornar o sistema ainda mais eficiente.

Essa lógica local será combinada com a transmissão seletiva via MQTT, de modo que o sistema envie apenas os eventos relevantes para o dashboard. Na próxima subseção, será detalhada a experiência prática.

3.2 Experiência prática

Nesta experiência prática, adota-se a abordagem de Edge Computing para demonstrar como o processamento pode ser realizado diretamente no microcontrolador, sem a necessidade de envio constante de dados para servidores externos. Utilizando o ESP32 conectado ao sensor inercial MPU6050, o objetivo é detectar eventos significativos de movimento diretamente na borda, ou seja, no próprio dispositivo embarcado.

Disponibilizamos o código para que você possa ir além e simular o sensor MPU6050. Você encontrará esse código na seção Assets desta fase, chamado “MPU6050_INT.ino”. Há três opções para executar esse código: no VSCode (seguindo os passos de instalação do Wokwi e Platformio, disponíveis no vídeo HOW TO: Como usar o Wokwi no VSCode), no Wokwi Web (wokwi.com) ou, até mesmo,

adquirindo seu próprio kit de hardware. Busque na internet por kits Arduinos que tenham o sensor MPU6050 ou peça sugestões de kits ao tutor.

Diferente das experiências anteriores com Cloud e Fog Computing, nas quais os dados brutos eram transmitidos em tempo real para análise posterior, nesta proposta o próprio firmware embarcado no ESP32 é responsável por interpretar os dados do acelerômetro. Caso um evento seja identificado — como, por exemplo, uma aceleração linear maior que um determinado limiar —, o microcontrolador aciona uma rotina de interrupção e envia uma notificação via MQTT. Isso significa que apenas dados realmente relevantes são transmitidos, reduzindo o consumo de energia, o uso da rede e o processamento externo.

Para isso, o pino de interrupção do MPU6050 é utilizado para indicar ao ESP32 que um evento de aceleração foi detectado. A lógica de decisão é programada diretamente no firmware, por meio de filtros e condições baseadas nos valores de aceleração capturados. O envio da mensagem via MQTT ocorre apenas após a confirmação do evento, tornando a comunicação mais eficiente e enxuta.

A visualização desses eventos é realizada por meio de um dashboard minimalista no Node-RED, que exibe somente os alertas enviados. Com isso, elimina-se a sobrecarga de dados e permite-se uma arquitetura de resposta rápida e autônoma.

Na próxima subseção, serão apresentados os detalhes do hardware utilizado, incluindo o uso do pino de interrupção do sensor MPU6050 conectado ao ESP32.

3.2.1 Hardware – ESP32 e MPU6050 com interrupção no pino INT

A montagem física do hardware nesta experiência é semelhante à utilizada nas abordagens de Cloud e Fog Computing, mantendo a combinação do microcontrolador ESP32 com o sensor inercial MPU6050. A principal diferença está no uso do pino de interrupção do sensor, que permite uma comunicação mais eficiente entre os dispositivos, acionando a leitura apenas quando um evento relevante for detectado.

No contexto da computação na borda, esse pino de interrupção (INT) desempenha um papel fundamental. Ao ser ativado, ele sinaliza ao ESP32 que uma aceleração acima do limiar configurado ocorreu, evitando que o microcontrolador fique

constantemente fazendo leituras do sensor. Isso reduz significativamente o consumo de energia e otimiza a performance do sistema.

O quadro a seguir apresenta os pinos utilizados na conexão entre o ESP32 e o MPU6050, incluindo a adição do pino INT para interrupções.

Função	Pino no MPU6050	Pino no ESP32
Alimentação (3.3V ou 5V)	VCC	3V3 ou VIN
Terra	GND	GND
Comunicação I2C (dados)	SDA	GPIO 21
Comunicação I2C (clock)	SCL	GPIO 22
Endereço I2C	AD0 (opcional)	GND
Interrupção	INT	GPIO 15

Quadro 1 - Conexões entre ESP32 e MPU6050 com uso do pino de interrupção
Fonte: Elaborado pelo autor (2025)

A alimentação do MPU6050 pode ser feita tanto com 3.3V quanto com 5V, dependendo da versão do sensor utilizada. O barramento I2C é composto pelos pinos SDA e SCL, e o pino AD0 pode ser mantido em GND para usar o endereço padrão do dispositivo. O pino INT é conectado ao GPIO 15 do ESP32 e será configurado no firmware para responder a eventos de aceleração definidos por limiares programados.

Essa configuração de hardware permite que o microcontrolador reaja de forma autônoma e em tempo real a estímulos físicos, estabelecendo uma base sólida para a lógica embarcada que será apresentada na próxima subseção.

3.2.2 Processamento embarcado – Filtragem e tomada de decisão no firmware

Nesta etapa da experiência, o foco está na inteligência embarcada diretamente no microcontrolador. Em vez de transmitir continuamente os dados do sensor MPU6050, o ESP32 executa a lógica de decisão local, com base em limiares de aceleração. O objetivo é identificar movimentos bruscos e enviar notificações apenas quando eventos relevantes forem detectados.

Neste exemplo, optou-se por utilizar o recurso de interrupção via pino INT do MPU6050 como mecanismo de detecção de eventos. Essa abordagem permite que o sensor notifique o ESP32 automaticamente quando a aceleração ultrapassa um valor determinado, eliminando a necessidade de leituras contínuas no loop principal. No entanto, é importante destacar que existem outras formas de resolver esse problema, como, por exemplo, realizar leituras periódicas e análise por software, detecção de picos por algoritmos embarcados, filtros digitais no firmware ou uso de janelas móveis de aceleração. A escolha da estratégia depende dos requisitos do projeto, como consumo energético, tempo de resposta e capacidade de processamento.

A seguir, apresenta-se a implementação utilizando interrupções. O primeiro passo consiste na configuração da interrupção do sensor MPU6050. Essa funcionalidade é habilitada via registradores internos do sensor, permitindo que o pino INT seja acionado sempre que a aceleração ultrapassar um valor determinado. No firmware, essa interrupção é capturada por uma função de callback, que é executada imediatamente quando o evento ocorre.

```
mpu.setMotionDetectionThreshold(2); // Sensibilidade do
evento
mpu.setMotionDetectionDuration(5);  // Duração mínima do
movimento
mpu.setInterruptLatch(0);           // Modo de reinício
automático
mpu.setInterruptMode(1);            // Interrupção em
nível alto
mpu.setMotionInterrupt(true);       // Ativa interrupção
por movimento


pinMode(INT_PIN, INPUT);
attachInterrupt(digitalPinToInterrupt(INT_PIN),
handleMotionInterrupt, RISING);
```

Código-fonte 5 - Inicialização do sensor MPU6050 com interrupção configurada
Fonte: Elaborado pelo autor (2025)

Neste trecho, a função setMotionDetectionThreshold define a sensibilidade mínima da aceleração para disparar a interrupção. O comando attachInterrupt vincula o pino INT a uma função de interrupção no ESP32, permitindo uma resposta rápida ao evento detectado pelo sensor.

A função de tratamento da interrupção é bastante simples. Ao ser chamada, ela apenas marca uma variável que será utilizada na rotina principal do loop para confirmar que um evento deve ser processado.

```
volatile bool movimentoDetectado = false;

void handleMotionInterrupt() {
  movimentoDetectado = true;
}
```

Código-fonte 6 - Função de interrupção para tratamento do evento
Fonte: Elaborado pelo autor (2025)

No loop(), o código verifica se a variável movimentoDetectado foi ativada e, em caso positivo, realiza a leitura dos valores de aceleração e envia a mensagem via MQTT. Em seguida, a variável é redefinida para aguardar o próximo evento.

```
void loop() {
  if (movimentoDetectado) {
    mpu.getAcceleration(&ax, &ay, &az);

    String payload = "{\"ax\":" + String(ax) + ",\"ay\":"
+ String(ay) + ",\"az\":" + String(az) + "}";
    client.publish("sensor/movimento", payload.c_str());

    movimentoDetectado = false;
  }

  delay(100);
}
```

Código-fonte 7 - Loop principal com lógica de decisão
Fonte: Elaborado pelo autor (2025)

Esse modelo de decisão embarcada é altamente eficiente para aplicações de detecção de anomalias, vibração, quedas ou movimentações específicas, uma vez que minimiza o tráfego de dados e prioriza eventos críticos. A lógica local garante resposta imediata sem depender de conectividade constante com a internet.

Na próxima subseção, será apresentada a estratégia de comunicação eficiente via MQTT, destacando como a arquitetura embarcada envia apenas os dados filtrados ao broker local, otimizando os recursos do sistema.

3.2.3 Comunicação eficiente – Envio apenas de eventos relevantes via MQTT

Uma das maiores vantagens da abordagem de Edge Computing é a eficiência na comunicação. Em vez de transmitir todos os dados coletados pelo sensor, o dispositivo realiza um pré-processamento e envia apenas as informações consideradas relevantes. Isso reduz drasticamente o volume de dados trafegados, o consumo de energia e o uso de recursos computacionais em sistemas posteriores.

Nesta experiência, essa estratégia foi implementada utilizando o protocolo MQTT para transmitir somente os eventos detectados pelo sensor MPU6050, após uma lógica de decisão embarcada no ESP32. Conforme descrito anteriormente, o envio é disparado apenas quando a interrupção por aceleração é acionada e os dados de movimento são confirmados como significativos.

A publicação MQTT segue a mesma lógica utilizada nas experiências anteriores, porém agora ocorre sob demanda, quando há um evento relevante. Isso significa que, durante períodos de inatividade ou movimentações irrelevantes, o microcontrolador permanece em estado ocioso, economizando energia e recursos de rede.

```
if (movimentoDetectado) {
  mpu.getAcceleration(&ax, &ay, &az);

  String payload = "{\"ax\":" + String(ax) + ",\"ay\":" +
String(ay) + ",\"az\":" + String(az) + "}";
  client.publish("sensor/movimento", payload.c_str());

  movimentoDetectado = false;
}
```

Código-fonte 8 - Publicação MQTT somente após evento de aceleração
Fonte: Elaborado pelo autor (2025)

O payload MQTT é estruturado em formato JSON, contendo os valores de aceleração nos três eixos. O tópico utilizado é "sensor/movimento", mas pode ser

adaptado conforme a arquitetura desejada. A publicação ocorre utilizando a biblioteca MQTT padrão, conectada a um broker local (por exemplo, Mosquitto), previamente configurado na rede.

Esse modelo de envio por eventos é altamente escalável, sendo ideal para aplicações que exigem resposta rápida, economia de bateria e operação em ambientes com restrição de largura de banda. Ele também reduz o esforço necessário para análise posterior, já que o sistema recebe apenas registros relevantes.

Na próxima subseção, será apresentada a visualização desses dados por meio de uma interface simplificada no Node-RED, que exibe em tempo real os alertas de movimento detectados, completando a arquitetura de Edge Computing.

3.2.4 Visualização – Interface mínima com Node-RED

Com o processamento de dados realizado diretamente no microcontrolador e a transmissão otimizada via MQTT, a visualização também pode ser simplificada. Nesta experiência, utilizamos uma interface mínima construída no Node-RED para indicar apenas os eventos de movimento detectados e transmitidos pelo ESP32.

A próxima figura ilustra o fluxo implementado no Node-RED. O nó MQTT Movimento assina o tópico "sensor/movimento" e recebe os dados apenas quando o evento é considerado relevante e enviado pelo microcontrolador. Em seguida, o nó “Detecta e liga LED” interpreta a mensagem e aciona um LED virtual (indicador visual no painel do Node-RED) para representar a ocorrência de movimento.

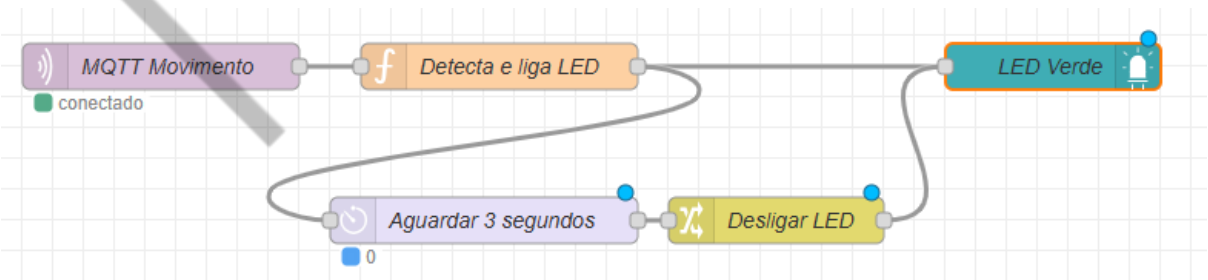


Figura 8 - Fluxo no Node-RED para visualização mínima dos eventos de movimento
Fonte: NODE-RED (2025)

O “LED Verde” acende por três segundos sempre que um evento de movimento é detectado, graças à lógica de temporização presente no nó para “Aguardar 3 segundos” seguido do nó “Desligar LED”, que apaga o indicador ao final do intervalo.

Esse comportamento garante uma visualização clara, sem a necessidade de gráficos contínuos ou monitoramento constante de dados.

Esse tipo de dashboard é ideal para aplicações em que o objetivo é apenas registrar a ocorrência de eventos, como em sistemas de alarme, notificações ou detecção de anomalias. Sua leveza e simplicidade tornam a interface adequada a ambientes com recursos limitados e foco em eficiência.

Na próxima seção, será realizada uma comparação entre as arquiteturas Cloud, Fog e Edge, destacando os pontos fortes e limitações de cada uma, além de oferecer critérios práticos para escolha da abordagem ideal em projetos de IoT.

FIAP

4 COMPARAÇÃO ENTRE NUVEM, NÉVOA E BORDA

As três abordagens de processamento distribuído em IoT, Cloud, Fog e Edge Computing, possuem o objetivo comum de coletar, processar e disponibilizar informações provenientes de dispositivos conectados. Apesar dessa convergência, cada uma delas apresenta diferenças importantes relacionadas ao local onde o processamento é realizado, à forma de transmissão e armazenamento de dados e aos requisitos de infraestrutura necessários para sua operação.

A Computação em Nuvem concentra o processamento e o armazenamento em data centers remotos, com alta capacidade de processamento, redundância e escalabilidade praticamente ilimitada. Essa abordagem é adequada para análises históricas, processamento intensivo e integração com sistemas corporativos de grande porte. Entretanto, a dependência da conectividade com a internet pode comprometer o desempenho em aplicações que demandam baixa latência.

A Computação em Névoa funciona como uma camada intermediária entre a borda e a nuvem. O processamento é realizado mais próximo da origem dos dados, mas fora do dispositivo final. Essa arquitetura geralmente é implementada em gateways, servidores locais ou nós distribuídos que recebem, agregam e filtram os dados antes de enviá-los à nuvem. Com isso, há redução no volume de dados transmitidos, melhor aproveitamento da largura de banda e menor tempo de resposta para aplicações em tempo quase real.

A Computação de Borda leva o processamento para o ponto mais próximo da geração dos dados, muitas vezes no próprio microcontrolador ou dispositivo embarcado. Essa proximidade reduz significativamente a latência, possibilita decisões instantâneas e aumenta a resiliência do sistema, já que sua operação não depende de conexão constante com redes externas. Em contrapartida, essa abordagem apresenta limitações de capacidade de processamento e armazenamento quando comparada às outras camadas.

Compreender as particularidades de cada modelo é essencial para escolher a estratégia mais eficiente para um projeto de IoT. Para apoiar essa análise, a próxima subseção apresenta uma matriz comparativa detalhada que sintetiza as características técnicas, vantagens e limitações das três abordagens, facilitando a identificação da mais adequada para diferentes cenários.

4.1 Matriz comparativa detalhada

A Computação em Nuvem realiza o processamento principalmente em data centers remotos, hospedados por provedores públicos ou privados. Esse modelo oferece alta capacidade de processamento e armazenamento praticamente ilimitado, sendo ideal para aplicações que demandam análise de grandes volumes de dados, como Big Data, Inteligência Artificial complexa e integração corporativa em larga escala. No entanto, apresenta latência elevada devido à distância física e ao tempo necessário para transmitir dados brutos pela internet. O consumo de largura de banda também é alto, já que grandes quantidades de informações precisam ser enviadas integralmente. Em termos de segurança, embora os data centers adotem políticas robustas e infraestrutura protegida, os dados permanecem vulneráveis durante o trajeto até a nuvem. A resiliência a falhas de conexão é baixa, pois a operação depende fortemente de internet estável. A manutenção é centralizada e realizada pelo provedor, e o custo pode variar conforme o volume de dados processados e o nível de serviço contratado.

A Computação em Névoa atua como camada intermediária entre a nuvem e os dispositivos de borda, utilizando servidores locais ou gateways próximos à origem dos dados. Essa abordagem reduz a latência para níveis médios, pois parte do processamento e filtragem ocorre antes do envio das informações para a nuvem. O consumo de largura de banda também é moderado, graças ao pré-processamento que descarta dados irrelevantes. A capacidade de processamento é média a alta, dependendo da potência dos nós intermediários, e a resiliência a falhas é superior à da nuvem, já que algumas funções podem continuar operando localmente mesmo sem conexão.

No aspecto de segurança, a proximidade com a fonte permite aplicar criptografia e autenticação logo no início do fluxo, diminuindo riscos. O armazenamento é intermediário e geralmente temporário, com envio periódico para a nuvem. A manutenção é parcialmente centralizada, mas exige atenção aos gateways distribuídos. O custo de implementação é médio, devido à necessidade de infraestrutura adicional. É amplamente utilizada em monitoramento industrial, cidades inteligentes e sistemas de controle de tráfego.

A Computação na Borda leva o processamento para o próprio dispositivo embarcado ou microcontrolador. Essa abordagem oferece latência extremamente baixa, possibilitando respostas quase instantâneas e consumo mínimo de largura de banda, já que apenas eventos relevantes ou dados processados são transmitidos. No entanto, a capacidade de processamento é limitada pelas restrições de hardware. A resiliência a falhas de conexão é alta, pois os dispositivos podem operar de forma autônoma mesmo sem acesso à rede externa. Em termos de segurança, as decisões locais reduzem a exposição de dados, embora a proteção dependa da robustez das bibliotecas e recursos embarcados. O armazenamento é restrito, sendo utilizado principalmente para logs e dados temporários. A manutenção é descentralizada, exigindo atualizações individuais em cada unidade, o que pode aumentar o custo operacional em grandes implantações. O custo por dispositivo tende a ser baixo, mas o gerenciamento de larga escala pode elevar os investimentos. Essa abordagem é especialmente indicada para automação residencial, robótica, veículos autônomos e sistemas de resposta imediata.

O próximo quadro sintetiza essas características de forma objetiva, apresentando uma visão comparativa das três arquiteturas em relação a local de processamento, latência, consumo de largura de banda, capacidade de processamento, resiliência a falhas, segurança, armazenamento, manutenção, custo e casos de uso típicos. Essa análise evidencia que Cloud, Fog e Edge Computing não competem diretamente, mas se complementam em arquiteturas IoT híbridas capazes de equilibrar desempenho, custo e segurança.

Característica	Cloud Computing	Fog Computing	Edge Computing
Local de processamento	Data centers remotos	Gateways e servidores locais	Dispositivo embarcado
Latência	Alta	Média	Muito baixa
Consumo de largura de banda	Alto	Médio	Muito baixo
Capacidade de processamento	Muito alta	Média/Alta	Limitada
Resiliência a falhas de conexão	Baixa	Média	Alta
Segurança	Alta no data center, vulnerável no trajeto	Boa, com criptografia próxima da fonte	Alta, com baixa exposição externa
Armazenamento	Ilimitado	Temporário/Intermediário	Limitado
Manutenção	Centralizada	Parcialmente centralizada	Descentralizada
Custo	Variável (por uso)	Médio	Baixo por unidade, alto em larga escala
Casos de uso típicos	Big Data, IA complexa, integração corporativa	Monitoramento industrial, cidades inteligentes, controle de tráfego	Automação residencial, robótica, veículos autônomos

Quadro 2 - Comparação objetiva entre Cloud, Fog e Edge Computing
Fonte: Elaborado pelo autor (2025)

Essa compreensão fornece a base necessária para avançarmos à próxima subseção, onde serão examinados de forma detalhada o consumo de energia, a latência e a robustez dessas três abordagens, permitindo uma escolha técnica mais fundamentada para cada cenário de aplicação.

4.2 Análise de consumo de energia, latência e robustez

A avaliação comparativa entre computação em nuvem, névoa e borda, quando limitada aos parâmetros de consumo de energia, latência e robustez, permite identificar de forma objetiva os contextos nos quais cada abordagem se destaca ou apresenta limitações. Esses três fatores são críticos em aplicações IoT, pois influenciam diretamente a eficiência operacional, a experiência do usuário e a viabilidade de longo prazo de um projeto.

No que diz respeito ao consumo de energia, a computação em nuvem tende a exigir mais dos dispositivos de campo. Isso ocorre porque todo o fluxo de dados, muitas vezes em grandes volumes, precisa ser transmitido continuamente pela rede até os data centers, elevando o uso dos módulos de comunicação, que são os principais responsáveis pelo gasto energético em dispositivos IoT. Em contrapartida, a computação em névoa reduz essa demanda por meio do pré-processamento em gateways locais, enviando para a nuvem apenas informações relevantes ou já agregadas. A computação na borda, por sua vez, praticamente elimina o custo energético com transmissões frequentes, pois o envio de dados ocorre apenas em eventos críticos ou sob demanda. No entanto, o processamento local intensivo pode elevar o consumo interno do microcontrolador ou sistema embarcado.

A latência é outro fator determinante. Em sistemas baseados exclusivamente na nuvem, a distância física e o roteamento dos pacotes até servidores remotos podem resultar em tempos de resposta que variam de dezenas a centenas de milissegundos, inviabilizando aplicações críticas em tempo real. A camada de névoa melhora significativamente esse cenário ao manter parte do processamento próximo à origem dos dados, reduzindo a latência para a casa de alguns milissegundos. Já na borda, a latência pode atingir valores sub-milisegundo, permitindo respostas praticamente instantâneas e essenciais para sistemas autônomos ou de segurança.

Por fim, a robustez está diretamente associada à capacidade de operação sob falhas de conexão ou interrupções de serviço. A arquitetura em nuvem apresenta menor robustez, pois a dependência de uma conexão contínua e estável é total. A camada de névoa oferece resiliência intermediária, já que funções básicas podem continuar sendo executadas nos gateways mesmo sem internet. A borda é a mais robusta nesse quesito, permitindo que dispositivos operem de forma completamente autônoma, mantendo funcionalidades críticas mesmo em isolamento prolongado.

Para facilitar a visualização desses aspectos, o quadro a seguir apresenta um resumo técnico dos três parâmetros analisados, permitindo rápida correlação entre arquitetura e desempenho esperado.

Arquitetura	Consumo de Energia	Latência	Robustez
Cloud Computing	Alto consumo nos dispositivos devido à necessidade de transmissão contínua de grandes volumes de dados pela rede.	Alta latência (dezenas a centenas de ms) devido à distância física até o data center e tempo de roteamento.	Baixa – depende de conexão estável; falha de internet interrompe a operação.
Fog Computing	Consumo moderado, pois o pré-processamento nos gateways reduz o volume e a frequência das transmissões.	Latência média (alguns ms) graças ao processamento próximo à origem dos dados.	Média – parte das funções pode continuar operando localmente em caso de falha de conexão.
Edge Computing	Consumo mínimo em transmissões, mas com possível aumento no consumo local devido ao processamento embarcado.	Latência muito baixa (sub-ms) com respostas praticamente instantâneas.	Alta – operação autônoma mesmo sem conexão externa.

Quadro 3 - Comparativo técnico entre Cloud, Fog e Edge Computing nos aspectos de consumo de energia, latência e robustez
Fonte: Elaborado pelo autor (2025)

4.3 Critérios para escolha da abordagem ideal

A seleção entre computação em nuvem, névoa e borda, ou a combinação dessas arquiteturas em um modelo híbrido, exige uma análise criteriosa que considere simultaneamente requisitos técnicos, econômicos e operacionais. O desempenho e a viabilidade de um sistema IoT dependem da correta adequação entre as características da aplicação e as propriedades intrínsecas de cada abordagem.

O primeiro critério a ser avaliado é a latência aceitável para a aplicação. Em sistemas que exigem respostas imediatas, como controle de frenagem em veículos autônomos, robótica industrial de alta precisão ou dispositivos médicos críticos, a computação na borda é praticamente mandatória devido à sua capacidade de resposta em tempo real. Para cenários que demandam tempos de reação rápidos, mas não necessariamente instantâneos, como monitoramento de tráfego urbano ou controle de processos industriais não críticos, a camada de névoa é suficiente. Aplicações que toleram atrasos mais elevados, como análise de tendências de consumo ou processamento de grandes volumes de dados históricos, podem ser suportadas exclusivamente pela nuvem.

O segundo critério refere-se ao consumo energético e disponibilidade de recursos locais. Dispositivos alimentados por bateria ou energia colhida do ambiente (*energy harvesting*) devem minimizar transmissões contínuas, favorecendo o processamento na borda ou na névoa. Por outro lado, equipamentos conectados a uma fonte de energia estável e com requisitos de processamento pesado, como análise preditiva baseada em Inteligência Artificial, podem se beneficiar da capacidade de processamento escalável da nuvem.

O terceiro critério é a robustez operacional e tolerância a falhas de conectividade. Em ambientes onde a conexão é instável ou sujeita a interrupções, como áreas rurais, operações em alto-mar ou missões críticas em zonas de desastre, a capacidade de manter funções autônomas torna a borda e a névoa alternativas indispensáveis. A nuvem pode complementar a arquitetura para sincronização e backup quando a conexão for restabelecida.

O quarto critério é a complexidade de manutenção e custo total de propriedade (TCO). A computação em nuvem centraliza a manutenção, reduzindo a carga operacional sobre o usuário, mas pode gerar custos recorrentes elevados devido ao consumo de largura de banda e processamento remoto. A borda, embora reduza custos de transmissão, requer manutenção distribuída em cada dispositivo, o que aumenta o esforço logístico. A névoa, por sua vez, apresenta um equilíbrio, com manutenção intermediária concentrada nos gateways.

O último critério está relacionado à segurança e conformidade regulatória. Em setores regulados, como saúde, finanças e defesa, pode ser obrigatório manter dados sensíveis o mais próximo possível de sua origem, reduzindo riscos de interceptação durante a transmissão. Nesse contexto, a borda e a névoa oferecem vantagens significativas, pois permitem criptografia e autenticação já nos estágios iniciais do fluxo de dados.

Para facilitar a aplicação desses critérios, o quadro a seguir apresenta um resumo que relaciona cada requisito a uma recomendação arquitetural predominante, sem excluir a possibilidade de combinações híbridas.

Critério	Cloud Computing	Fog Computing	Edge Computing
Latência mínima necessária.	Baixa prioridade.	Média prioridade.	Alta prioridade.
Consumo de energia reduzido.	Baixa prioridade.	Média prioridade.	Alta prioridade.
Operação autônoma sem rede.	Baixa prioridade.	Média prioridade.	Alta prioridade.
Manutenção centralizada.	Alta prioridade.	Média prioridade.	Baixa prioridade.
Processamento intensivo de dados.	Alta prioridade.	Média prioridade.	Baixa prioridade.
Proteção de dados sensíveis.	Média prioridade.	Alta prioridade.	Alta prioridade.

Quadro 4 - Critérios de seleção e recomendação de abordagem predominante
Fonte: Elaborado pelo autor (2025)

Ao aplicar os critérios apresentados, torna-se evidente que a decisão mais eficiente, na maioria dos casos, envolve a integração das três camadas em uma arquitetura híbrida, onde a borda lida com respostas imediatas e filtragem inicial, a névoa realiza o processamento intermediário e a agregação de dados, e a nuvem concentra o armazenamento massivo e a análise avançada. Essa sinergia garante um equilíbrio entre desempenho, robustez e custo, maximizando a eficiência global do sistema IoT.

Com esta base, conclui-se a análise comparativa e de seleção de arquitetura, criando o ponto de transição natural para a conclusão do capítulo, onde serão sintetizados os aprendizados e destacadas as diretrizes para projetos futuros.

REFERÊNCIA

NODE-RED. Plataforma de desenvolvimento visual para lógica de aplicações IoT. **Node-RED**, 2025. Disponível em: <https://nodered.org/>. Acesso em: 02 set. 2025.

OLIVEIRA, Bruno. Arquitetura de computação em névoa. Internet das Coisas, **Medium**, 15 de junho de. 2019. Disponível em: <https://medium.com/internet-das-coisas/fog-03-arquitetura-de-computa%C3%A7%C3%A3o-em-n%C3%A9voa-69b2d8bbf1f7>. Acesso em: 02 set. 2025.

FIAP